

Module 05: Classes and Objects

This is graduation day. Every file so far has been a compact source file. From here on the examples are written the classic way, because that is what every real codebase, tutorial and Stack Overflow answer uses.

Nothing new is happening to the language. Module 01 used `javap` to show the compiler generating a class for you all along. Now you write it yourself.

What You'll Learn

- Every word in `public static void main(String[] args)`, and why each is there
- Fields, the four visibility levels, and why `private` is where you start
- Constructors, `this(...)`, and the free constructor you lose by writing one
- Why static factory methods often beat constructors
- Flexible constructor bodies, which let you validate before `super()`

Coming From Another Language

	Python	Java
Constructor	<code>__init__</code>	a method named after the class
Current instance	explicit <code>self</code> parameter	implicit <code>this</code>
Privacy	<code>_name</code> by convention	<code>private</code> , enforced by the compiler
Multiple constructors	one <code>__init__</code> , default args	overload, or static factories
Class-level data	class attribute	<code>static</code> field
File layout	many classes per file, freely	one <code>public</code> class per file, named to match

The privacy row is the real difference. Python's underscore is a request. Java's `private` is a rule the compiler enforces, and code that violates it does not build.

The Lesson

The classic form, word by word

```
public class TheClassicForm {  
    public static void main(String[] args) {  
        System.out.println("Running the classic way.");  
    }  
}
```

Word	Why
<code>public</code> (class)	usable from anywhere. Must match the file name.
<code>public</code> (main)	the JVM has to be able to call it
<code>static</code>	the important one. At startup no object exists yet, so <code>main</code> cannot belong to one.
<code>void</code>	returns nothing; exit codes come from <code>System.exit</code>
<code>main</code>	the exact name the JVM looks for
<code>String[] args</code>	command-line arguments. Never null; empty when there are none.

`this` does not exist in `main`, because it is static. To use instance state, `main` constructs an object first, which is what most real `main` methods do.

Which form should you use? For learning, scripts and single-file tools, the compact form is less noise. For anything multi-file, a library, or code other people touch, use the classic form. They are the same language; you are choosing how much to type.

Encapsulation

Four visibility levels, narrowest first:

Level	Reach
<code>private</code>	this class only
<i>(none)</i>	this package. Called package-private.
<code>protected</code>	this package, plus subclasses anywhere
<code>public</code>	everyone

Start at `private` and widen when you have a reason. Widening later is easy. Narrowing later breaks everyone who depended on it.

The point is not secrecy, it is **invariants**. From `Encapsulation.java`:

```
refused: insufficient funds: have 15000, asked for 100000  
balance is still: 15000
```

Because `balanceInCents` is private, every change goes through a method that enforces the rules, so the class can promise the balance is never negative and actually keep that promise. A class with public mutable fields has no invariants at all, since anything can change them at any time.

`final` on a field means assigned exactly once, in the constructor. Note that this is not the same as immutable: a `final` field can still point at a mutable object.

Constructors

Writing any constructor removes the free one. Java supplies a no-argument constructor only if you declare none at all. This is why adding a constructor to an existing class can break callers who wrote `new Thing()`.

`this(...)` delegates to another constructor:

```
Temperature() {  
    this(0.0);  
}
```

Static factories often beat constructors because they have names:

```
Temperature.fromFahrenheit(77.0)  
Temperature.fromKelvin(298.15)
```

Two constructors both taking a single `double` cannot coexist. Two factories can. A factory can also return a cached instance or a subclass, where a constructor always builds something new.

Fields get defaults, locals do not. A field is `0`, `false`, `null` or `0.0` automatically. A local variable used before assignment is a compile error. The asymmetry is deliberate: a field may legitimately be set later by another method, so the compiler cannot prove anything, while a local is used a few lines from its declaration, so the compiler can check and does.

Flexible constructor bodies

The old rule: `super(...)` had to be the very first statement. You constructed the parent, then discovered the arguments were invalid.

JEP 513, final in JDK 25, lets statements run first:

```
Positive(int value) {  
    if (value <= 0) {  
        throw new IllegalArgumentException("must be positive, got " + value);  
    }  
    super(value);  
}
```

`FlexibleConstructorBodies.java` output:

```
Measurement constructor ran with 5
built: 5

now with an invalid value:
rejected: must be positive, got -1
```

Read that carefully. The parent constructor ran **once**, for the valid case. For the invalid one it never ran at all, because the check threw first. Under the old rule that was impossible without smuggling the check into a static helper inside the `super()` call.

`this` is still off limits before `super()`, because the parent has not run and the object is not yet valid. The compiler enforces that.

Run It

```
java modules/05-classes-and-objects/examples/TheClassicForm.java
java modules/05-classes-and-objects/examples/TheClassicForm.java hello world
java modules/05-classes-and-objects/examples/Encapsulation.java
java modules/05-classes-and-objects/examples/Constructors.java
java modules/05-classes-and-objects/examples/FlexibleConstructorBodies.java

./scripts/verify-examples.sh modules/05-classes-and-objects
```

Common Mistakes

Public mutable fields. The class can no longer promise anything about its own state.

Adding a constructor and breaking `new Thing()` elsewhere. The free no-argument constructor vanishes the moment you declare any constructor.

Returning your internal collection from a getter. The caller can now modify your object's state without going through you. Module 04's homework covered the fix.

Assuming `final` means immutable.

```
private final List<String> items = new ArrayList<>();
items.add("still allowed"); // fine, the reference never changed
```

`final` fixes the reference, not the object.

A constructor that does real work. Constructors that open files or make network calls are hard to test and fail in ways callers cannot handle. Build the object, then call a method.

Writing getters and setters for every field reflexively. A setter for every field is a public mutable field with extra steps. Add one when something actually needs to change that value from outside.

Key Takeaways

- `static` on `main` exists because no object exists at startup.
- The classic and compact forms are the same language, differing only in what you type. Use classic for anything multi-file or shared.
- Start fields at `private`. Widening is easy, narrowing breaks callers.
- Encapsulation buys invariants, not secrecy.
- Declaring any constructor removes the free no-argument one.
- Static factories have names, so they can overload where constructors cannot.
- Fields default, locals do not.
- Statements may now precede `super()`, so you can reject bad arguments before the parent object is ever built.

Homework

[homework/README.md](#)

Build a class that cannot be put into an invalid state, then prove it by trying. Reference solution in [solutions/05-classes-and-objects/](#).